

Python Practice Set – 100 Questions

A structured progression from revision to mini-projects. Solve in order. Questions marked [Interview Important] are common in interviews / build core fundamentals. Don't ask for solutions first — try, then ask for review/hints.

Level 1: Easy Revision (Q1–25)

Quick exercises across all topics covered so far.

1. Take a number as input and print whether it is even or odd. **[Interview Important]**
2. Swap two variables without using a third variable. **[Interview Important]**
3. Convert a temperature from Celsius to Fahrenheit.
4. Take a string and print it in reverse using slicing. **[Interview Important]**
5. Check if a string is a palindrome (basic, no functions). **[Interview Important]**
6. Count vowels in a given string. **[Interview Important]**
7. Take a sentence and capitalize the first letter of every word (without using `.title()`).
8. Given a list of numbers, find the maximum and minimum without using `max()/min()`. **[Interview Important]**
9. Given a list, remove all duplicate elements (basic approach). **[Interview Important]**
10. Sort a list of numbers in descending order without using `sort(reverse=True)`.
11. Create a tuple of 5 fruits and print the 2nd and last element using indexing.
12. Count how many times a given element appears in a tuple using `count()`.
13. Create a dictionary of 5 students with their marks; print all keys and values separately. **[Interview Important]**
14. Given a dictionary, check if a particular key exists using two different methods. **[Interview Important]**
15. Merge two dictionaries into one. **[Interview Important]**
16. Create two sets and find their union and intersection. **[Interview Important]**
17. Remove a specific element from a set and handle the case if it doesn't exist.
18. Print all numbers from 1 to 50 that are divisible by 3 or 5.
19. Use a while loop to print the multiplication table of a given number.
20. Write a function that takes two numbers and returns their sum, difference, product and quotient. **[Interview Important]**
21. Write a function with a default parameter that greets a person by name (default 'Guest').
22. Write a function to check if a number is prime. **[Interview Important]**

23. Open a text file, write 3 lines into it, then read and print all lines. **[Interview Important]**
24. Write a program that takes a number as input and handles ValueError if the input is not a number. **[Interview Important]**
25. Use random.randint() to simulate rolling two dice and print their sum.

Level 2: Logic Building (Q26–55)

Focus on loops, functions, lists, dictionaries and sets — combining them in slightly tricky ways.

26. Print the following pattern using nested loops (for n=5 rows):
* * * * *
* * **[Interview Important]**
27. Print a number pyramid pattern: 1 1 2 1 2 3 1 2 3 4
28. Find all pairs of numbers in a list whose sum equals a given target (nested loop approach). **[Interview Important]**
29. Given a list of numbers, separate them into two lists: even numbers and odd numbers. **[Interview Important]**
30. Write a function that takes a list and returns a new list with only unique elements (no sets allowed). **[Interview Important]**
31. Find the second largest number in a list without sorting. **[Interview Important]**
32. Write a function to flatten a nested list (list containing lists) by one level using loops.
33. Count the frequency of each character in a string and store it in a dictionary. **[Interview Important]**
34. Write a function that takes a sentence and returns a dictionary with word frequency count. **[Interview Important]**
35. Given a dictionary of student names and marks, find the student with the highest marks. **[Interview Important]**
36. Given two lists, create a dictionary by zipping them as key-value pairs (without using zip()).
37. Write a function that merges two dictionaries; if a key exists in both, sum their values. **[Interview Important]**
38. Given a list of dictionaries (each representing a person with name and age), sort the list by age. **[Interview Important]**
39. Write a function that takes a list of numbers and returns a tuple of (sum, average, max, min).
40. Find common elements between two lists using sets, then using loops (compare both approaches). **[Interview Important]**
41. Write a function to check if two strings are anagrams of each other. **[Interview Important]**
42. Given a list of numbers, group them into a dictionary by even/odd as keys.

43. Write a nested loop program to print all multiplication tables from 1 to 5 (5x5 grid).
44. Write a function that takes a list of words and returns the longest word.
45. Given a string, remove all duplicate characters while preserving order. **[Interview Important]**
46. Write a function that counts how many times each element appears in a list and returns a dictionary. **[Interview Important]**
47. Implement a simple search: given a list of dictionaries (products with name and price), find a product by name.
48. Write a program to find all prime numbers in a given range using nested loops. **[Interview Important]**
49. Given a matrix (list of lists), find the sum of all elements using nested loops. **[Interview Important]**
50. Transpose a matrix (list of lists) using nested loops. **[Interview Important]**
51. Write a function that takes a dictionary and returns a new dictionary with keys and values swapped. **[Interview Important]**
52. Given a list of strings, group them in a dictionary by their length.
53. Write a program that removes all elements from a list that are also present in another list.
54. Write a function to check if a list contains any duplicate elements (return True/False), without using set(). **[Interview Important]**
55. Given a string, find the first non-repeating character. **[Interview Important]**

Level 3: Weakness Improvement (Q56–80)

Heavy focus on recursion, nested loops, and breaking problems into smaller functions/decomposition.

Recursion — Foundation Building

56. Write a recursive function to print numbers from 1 to n. **[Interview Important]**
57. Write a recursive function to print numbers from n to 1.
58. Write a recursive function to calculate the sum of digits of a number. **[Interview Important]**
59. Write a recursive function to count the number of digits in a number.
60. Write a recursive function to reverse a string. **[Interview Important]**
61. Write a recursive function to check if a string is a palindrome. **[Interview Important]**
62. Write a recursive function to find the power of a number (x^n) without using `**`. **[Interview Important]**
63. Write a recursive function to find the GCD of two numbers (Euclidean algorithm). **[Interview Important]**

- 64. Write a recursive function to find the nth Fibonacci number. **[Interview Important]**
- 65. Write a recursive function to find the maximum element in a list. **[Interview Important]**
- 66. Write a recursive function to count occurrences of a specific element in a list.
- 67. Write a recursive function to flatten a nested list of arbitrary depth (e.g. [1,[2,[3,4],5]]). **[Interview Important]**

Recursion — Thinking Practice (write the recurrence relation first, before coding)

- 68. Before coding: for 'sum of first n numbers', write down — what is the base case? what is the recursive case in terms of a smaller subproblem? Then code it. **[Interview Important]**
- 69. Write a recursive function to generate all subsets (power set) of a list. **[Interview Important]**
- 70. Write a recursive function to generate all permutations of a string. **[Interview Important]**
- 71. Write a recursive function for binary search on a sorted list. **[Interview Important]**

Nested Loops & Complex Logic

- 72. Print a hollow square pattern of stars for a given size n. **[Interview Important]**
- 73. Print a diamond pattern of stars for a given size n.
- 74. Given a 2D grid (list of lists) of 0s and 1s, count the number of 1s.
- 75. Given two matrices, write a program to add them element-wise using nested loops. **[Interview Important]**
- 76. Implement multiplication of two matrices using nested loops. **[Interview Important]**

Problem Decomposition (break into multiple small functions — this is the goal, not just the output)

- 77. Build a 'Number Analyzer': write separate functions is_even(), is_prime(), digit_sum(), and a main function that calls all three and prints a report for a given number. **[Interview Important]**
- 78. Build a 'Text Analyzer': write separate functions to count words, count vowels, find longest word, and check if it's a palindrome sentence — then combine in a main() function. **[Interview Important]**

Level 4: Mini Project Challenges (Q81–100)

Small programs that combine multiple concepts. Plan your functions/structure FIRST on paper before coding.

79. Contact Book (CLI): store contacts (name, phone, email) in a dictionary of dictionaries. Support add, view, search, delete, using functions for each operation. **[Interview Important]**
80. Inventory Management System: list of dictionaries for products (name, price, quantity). Functions to add stock, remove stock, search by name, and display low-stock items. **[Interview Important]**
81. Student Grade Manager: store student names and a list of marks in a dictionary. Functions to add a student, calculate average, assign grade (A/B/C...), and display a report. **[Interview Important]**
82. Word Frequency Counter from a File: read a text file, count frequency of each word (case-insensitive), and display the top 5 most common words. **[Interview Important]**
83. Simple To-Do List with File Persistence: tasks saved to a file; support add, view, mark complete, delete, with each operation as a separate function. **[Interview Important]**
84. Quiz Game: store questions, options and correct answers in a list of dictionaries; loop through, take user input, track score, and display result at the end with exception handling for invalid input. **[Interview Important]**
85. Bank Account Simulator: a dictionary of accounts (account number -> balance). Functions for deposit, withdraw (with balance check), transfer between accounts, and transaction history (stored in a list). **[Interview Important]**
86. Library Management Mini-System: dictionary of books (title -> {author, available: True/False}). Functions to issue a book, return a book, and search by author.
87. Recursive Directory-Style Printer: simulate a nested folder structure using nested dictionaries, and write a recursive function to print the structure with proper indentation. **[Interview Important]**
88. Palindrome & Anagram Checker Tool: combine string, recursion and set/dictionary logic — take two words and check if they are anagrams, and check if either is a palindrome (recursively).
89. Matrix Operations Toolkit: functions for addition, transpose, and a recursive function to compute the sum of all elements in a nested list (matrix). **[Interview Important]**
90. Tic-Tac-Toe (2-player, console-based): use a list of lists for the board, functions for displaying the board, checking valid moves, and checking for a winner. **[Interview Important]**
91. Password Strength Checker: function checks length, presence of digits, uppercase, lowercase and special characters; returns strength rating with reasons (combine string methods + conditionals + functions). **[Interview Important]**
92. Employee Management System with Sets: track employees and their skill-sets (dictionary of name -> set of skills). Functions to find employees with a given skill, and common skills between two employees using set operations.

93. Recursive Maze Path Counter: given a small grid (e.g. 4x4) with some blocked cells, write a recursive function to count the number of paths from top-left to bottom-right (only right/down moves). **[Interview Important]**
 94. Shopping Cart System: cart as a list of dictionaries (item, price, quantity); functions to add item, remove item, apply a discount, and calculate total bill with exception handling for invalid quantities. **[Interview Important]**
 95. File-based Expense Tracker: read/write expenses (date, category, amount) to a file; functions to add an expense, view all, and show total spent per category using a dictionary. **[Interview Important]**
 96. Recursive JSON-like Nested Dictionary Printer: given a deeply nested dictionary (mixed values and dictionaries), write a recursive function to print every key with its 'depth level' using indentation. **[Interview Important]**
 97. Number Guessing Game v2 (Enhanced): add difficulty levels (different ranges), limited attempts, hints (too high/low), and a function-based structure with exception handling for non-integer input.
 98. Capstone — Simple Library + Quiz Combo App: a menu-driven program (while loop) combining file handling, dictionaries, functions, recursion (at least one recursive helper), and exception handling — design your own feature set for a 'Student Helper App' that includes at least 3 of the above mini-project ideas as modules. **[Interview Important]**
-

How we'll work through this:

99. Solve questions in order — don't skip around too much, especially in Levels 3 and 4.
100. After you write code for a question, share it and I'll review + give hints (not full solutions) unless you explicitly ask for the solution.
101. For Level 3 recursion questions, try writing the base case and recursive case in plain English/comments BEFORE coding.
102. If stuck for more than ~15-20 minutes, ask for a hint rather than the answer — that's where the learning happens.